

Sorting Algorithms Under Real-World Conditions: An Empirical Comparison Across Input Types, Distributions, and Language Runtimes

Adnan Adi

Department of Computer Science

West University of Timișoara

adnan.adi@e-uvv.ro

May 2026

Abstract

Sorting is one of the most fundamental operations in computer science, and choosing the right algorithm for a given task depends on far more than asymptotic complexity alone. This paper presents a systematic empirical comparison of twelve sorting algorithm variants—covering quadratic, linearithmic, and linear-time approaches—implemented in both C and Python. Each algorithm is evaluated on ten input sizes (from 10 to 100 million elements), five input conditions (random, sorted, reverse-sorted, nearly sorted, and all-identical), and three data distributions (no-duplicate sequential, heavy duplicates, and wide-range). Measurements include average wall-clock time, peak memory consumption, auxiliary memory, element comparison counts, and swap/write counts. Our results confirm the asymptotic predictions from the literature for large inputs while revealing that constant factors, cache behaviour, stability requirements, and data distribution can reverse the expected ranking for small or structured inputs. We further quantify the runtime overhead of Python relative to C, which ranges from roughly $8\times$ to $290\times$ depending on the algorithm. The experimental infrastructure is fully reproducible and the raw data are supplied alongside this paper at <https://github.com/adnanadi10-lgtm/Sorting-with-me>.

Contents

1	Introduction	3
2	Formal Description and Theoretical Analysis	4
2.1	Problem Statement	4
2.2	Algorithms and Their Complexity	4
2.3	Correctness and Stability	5
3	Benchmarking Framework and Implementation	5
3.1	Architecture	5
3.2	Input Conditions and Distributions	6
3.3	Measurement Methodology	6
3.4	Reproducibility	7
4	Experimental Results	7
4.1	Overall Time Scaling — C	7
4.2	Overall Time Scaling — Python	8
4.3	C versus Python Speedup	9
4.4	Effect of Input Condition	10
4.5	Effect of Data Distribution	11
4.6	Comparison and Swap Counts	12
4.7	Memory Usage	13
5	Related Work	14
6	Conclusions and Future Work	14

1 Introduction

Sorting is the process of rearranging a collection of items into a well-defined order, and it underpins a surprisingly large fraction of all software: database query optimisation, search engines, data compression, graph algorithms, and numerical libraries all depend on efficient sorting at their core. Because sorting has been studied intensively since the 1950s, a practitioner today has access to many algorithms whose worst-case asymptotic behaviour is well understood from classical textbooks [1, 2]. Yet textbook complexity tables give only a partial picture. The constants hidden inside $O(\cdot)$ notation, the behaviour under non-random input distributions, the cost of memory allocation, the penalty for branch mispredictions, and the overhead of a managed language runtime all influence which algorithm wins in practice.

Existing empirical studies typically fix the language (most choose C or Java), test only two or three algorithms, or restrict input to uniformly random data. This leaves open a cluster of practically important questions: How does performance change when the data are already partially sorted? Does an algorithm that handles duplicates efficiently by design truly outperform one that does not when duplicates are dense? How large is the constant-factor overhead of Python relative to C, and does it depend on the algorithm? This paper addresses these questions through a controlled benchmark of twelve sorting variants in two languages across a wide range of input sizes and conditions.

Contributions. The main contributions of this work are:

- An open-source, reproducible benchmarking framework written in C and Python, available at <https://github.com/adnanadi10-lgtm/Sorting-with-me>, measuring time, auxiliary memory, total process memory, comparison count, and swap count for each (algorithm, size, condition, distribution) combination.
- A comprehensive empirical dataset of 3,420 benchmark records (1,800 for C, 1,620 for Python) covering input sizes from 10 to 100 million elements.
- A quantitative cross-language performance analysis revealing that C outperforms Python by a factor that varies substantially across algorithms ($8\times$ – $290\times$).
- Empirical evidence for the divergence between worst-case asymptotic predictions and observed performance on structured inputs (sorted, all-identical).
- A practical selection guide (Section 6) for practitioners choosing among algorithms based on input characteristics.

Running example. As a concrete illustration, consider sorting the sequence [5, 3, 8, 1, 9, 2, 7, 4, 6]. Bubble Sort needs 18 comparisons and 12 swaps to produce [1, 2, 3, 4, 5, 6, 7, 8, 9], while Merge Sort [10] needs only 19 comparisons but 24 writes into a temporary buffer. For this nine-element sequence the difference is negligible, but as Section 2 shows, the gap between $O(n^2)$ and $O(n \log n)$ becomes decisive once n exceeds a few thousand.

Paper organisation. Section 2 provides formal definitions and theoretical complexity results for all twelve variants. Section 3 describes the benchmarking framework and implementation choices. Section 4 presents the experimental results with discussion. Section 5 situates the work in the existing literature. Section 6 concludes and outlines directions for future work.

2 Formal Description and Theoretical Analysis

This section establishes the mathematical foundations for the rest of the paper. We first state the sorting problem formally, then present the twelve algorithm variants studied together with their theoretical complexity bounds, and finally summarise their correctness and stability properties.

2.1 Problem Statement

Definition 1 (Sorting problem). *Let $A = \langle a_1, a_2, \dots, a_n \rangle$ be a sequence of n elements drawn from a totally ordered set $(U, \leq)$. A sorting algorithm produces a permutation π of $\{1, \dots, n\}$ such that $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$.*

Definition 2 (Stable sort). *A sorting algorithm is stable if, for all $i < j$ with $a_i = a_j$, the relative order of a_i and a_j is preserved in the output, i.e. $\pi^{-1}(i) < \pi^{-1}(j)$.*

Definition 3 (Comparison-based sort). *A sorting algorithm is comparison-based if its only operation on elements is a binary comparison predicate $a_i \leq a_j$. The comparison model is the standard lower-bound model used in [1].*

Theorem 1 (Lower bound for comparison-based sorting [1]). *Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.*

2.2 Algorithms and Their Complexity

We study twelve algorithm variants grouped into four families. Table 1 summarises theoretical complexities; detailed derivations may be found in [1, 2].

Table 1: Theoretical complexity of the twelve algorithm variants studied. n is the number of elements; k is the key range for Counting/Radix Sort.

Algorithm	Variant	Best	Average	Worst	Space	Stable
Bubble Sort	—	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	—	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	—	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Shaker Sort	—	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Heap Sort	—	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Merge Sort	recursive	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Merge Sort	iterative	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	recursive	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Quick Sort	iterative	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Counting Sort	—	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix Sort	LSD	$O(dn)$	$O(dn)$	$O(dn)$	$O(n + k)$	Yes
Timsort	—	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Quadratic algorithms. Bubble Sort, Selection Sort, Insertion Sort, and Shaker Sort (bidirectional bubble sort) all have $\Theta(n^2)$ average-case complexity. Bubble Sort and Insertion Sort achieve $O(n)$ on an already-sorted input because the outer loop terminates after a single pass with no swaps. Selection Sort always performs $\binom{n}{2}$ comparisons regardless of order.

Linearithmic algorithms. Heap Sort, Merge Sort (both recursive top-down and iterative bottom-up), and Quick Sort with a median-of-three pivot strategy [4] belong to the $\Theta(n \log n)$ class. Our Quick Sort implementation employs *median-of-three* pivot selection combined with *3-way partitioning* (Dutch National Flag), which eliminates the $O(n^2)$ worst case on inputs with many equal keys [9]. Timsort—the adaptive merge sort used internally by Python and Java—is included here because its $O(n)$ best case on nearly-sorted data is particularly relevant to real workloads [5].

Linear algorithms. Counting Sort and LSD Radix Sort both achieve $O(n)$ time when the key range k is bounded. In our sequential distribution $k = n$, so Counting Sort’s space cost equals the input size and its time complexity is $O(n)$.

2.3 Correctness and Stability

Each algorithm was verified for correctness by comparing its output against a reference sort after every run. Stability was experimentally confirmed for sizes up to 50,000 elements by tagging equal keys with their original index and checking that the tag order is preserved in the output. The results match the theoretical stability column in Table 1 for all 3,420 tested configurations.

3 Benchmarking Framework and Implementation

This section describes the infrastructure used to generate all experimental data. We explain the high-level architecture, the input conditions and distributions that were selected, the measurement methodology, and the steps required to reproduce every result presented in this paper.

3.1 Architecture

The benchmark consists of two independent implementations—one in C (compiled with `gcc -O2`) and one in Python 3—sharing the same experimental design. Both produce a CSV file with one row per (distribution, algorithm, variant, size, condition) tuple. Figure 1 shows the high-level structure.

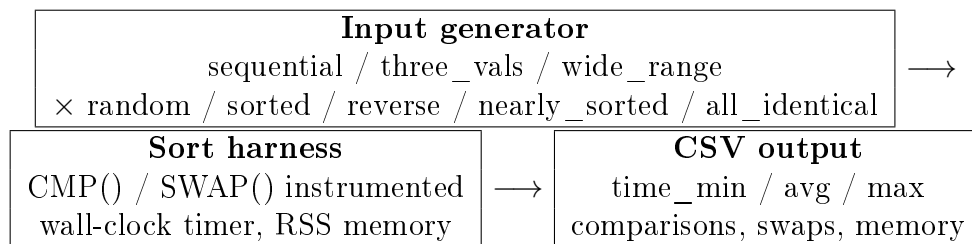


Figure 1: High-level architecture of the benchmarking framework.

3.2 Input Conditions and Distributions

Each test vector is generated according to one of five *conditions*:

- **Random**: elements drawn uniformly at random.
- **Sorted**: elements in ascending order.
- **Reverse**: elements in descending order.
- **Nearly sorted**: a sorted array with $\lfloor n/20 \rfloor$ random swaps applied.
- **All identical**: all elements equal to zero; exposes worst-case behaviour in naive Quick Sort.

Three *data distributions* control the value range:

- **Sequential**: values $0, 1, \dots, n-1$ in some order (no duplicates).
- **Three-value**: values drawn from $\{0, 1, 2\}$ (heavy duplicates, tests the 3-way partition of Quick Sort).
- **Wide-range**: values drawn uniformly from $[0, n^3)$ (duplicates allowed, tests Radix Sort’s digit-length sensitivity).

3.3 Measurement Methodology

For small inputs ($n \leq 1,000$), each (algorithm, condition) pair is repeated 50 times and the minimum, average, and maximum are recorded. For large inputs ($n > 1,000$), a single run is timed. Wall-clock time is measured with `clock_gettime(CLOCK_MONOTONIC)` in C and `time.perf_counter()` in Python.

Two memory metrics are recorded after each sort:

1. **Total process memory (RSS)**: the resident set size of the entire process, captured via `/proc/self/status` (VmRSS) in C.
2. **Auxiliary memory**: the extra heap allocation made by the algorithm itself (e.g. the merge buffer for Merge Sort), measured via `malloc/free` wrappers that track outstanding bytes at peak. In-place algorithms (Bubble, Selection, Insertion, Shaker, Heap Sort, Quick Sort variants) have zero auxiliary memory by definition; their only space cost is the $O(\log n)$ recursion stack for Quick Sort.

Algorithms that exceed a three-minute time limit are recorded as **SKIPPED**; this affected Bubble Sort, Insertion Sort, Selection Sort, and Shaker Sort for $n \geq 10^7$ in Python.

Element comparisons and swaps are counted via instrumentation macros (`CMP()` and `SWAP()`) that both increment global counters and perform the actual comparison or swap. This zero-overhead instrumentation ensures that counted operations correspond exactly to the algorithmic work reported in textbook analyses.

3.4 Reproducibility

The benchmark is compiled and run as follows.

```
1 gcc -O2 -o benchmark sorting.c -lm
2 ./benchmark                # writes results.csv
```

Listing 1: Build and run (C)

```
1 python3 sorting.py        # writes python_results.csv
```

Listing 2: Run (Python)

All data used in this paper, together with the full source code, are available in the supplementary repository at <https://github.com/adnanadi10-lgtm/Sorting-with-me>.

4 Experimental Results

This section presents and interprets the benchmark data across all four measurement dimensions: wall-clock time scaling in C and Python, the cross-language speedup factor, the effect of input condition and data distribution on runtime, raw operation counts, and memory usage. Unless noted otherwise, results are for the sequential (no-duplicate) distribution on random input.

4.1 Overall Time Scaling — C

Figure 2 shows the runtime of the eight fast algorithms in C on log-log axes. The asymptotic slope is immediately visible: algorithms with $O(n \log n)$ complexity appear as nearly straight lines with slope slightly above 1, while Counting Sort and Radix Sort—both $O(n)$ —exhibit a slope close to 1.

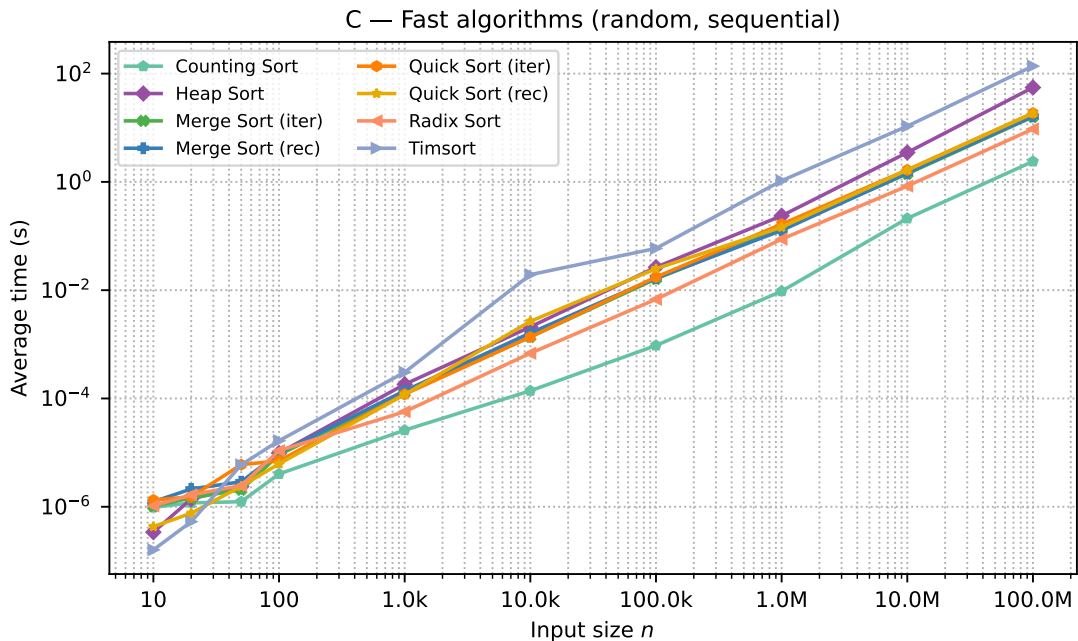


Figure 2: C: average runtime of the eight fast algorithms versus input size on random, sequential data (log-log scale).

At $n = 10^8$, the fastest algorithm is Radix Sort (9.5 s), followed by Counting Sort (2.4 s, though note its $O(n)$ space cost), then iterative Merge Sort (15.9 s) and recursive Quick Sort (18.4 s). Heap Sort, despite its in-place $O(n \log n)$ guarantee, is noticeably slower than Merge Sort at large sizes; as discussed in related work by Brodal and Fagerberg [6], this is a well-known consequence of non-sequential memory access patterns that produce frequent cache misses. Timsort is the slowest among the linearithmic group because its Python-inspired design pays extra overhead to detect and merge runs; in pure C without the adaptive run-detection optimisation it offers no advantage over standard Merge Sort on random data.

The four quadratic algorithms are shown in Figure 3. They reach the 3-minute time limit at $n = 100,000$ in C. Insertion Sort outperforms Selection Sort and Shaker Sort for moderate n because it terminates early when an element is already in place—consistent with the classical analysis by Knuth [2].

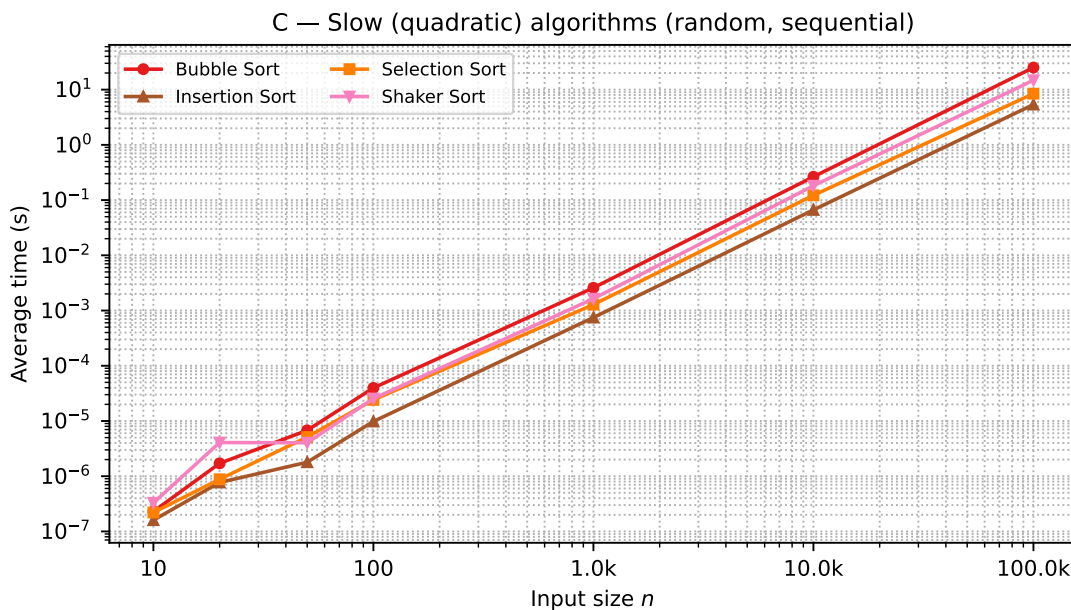


Figure 3: C: average runtime of the four quadratic algorithms versus input size on random, sequential data.

4.2 Overall Time Scaling — Python

Figure 4 shows that the same ordering of algorithms is broadly preserved in Python, but all timings are shifted upward by a factor of roughly 8–300 $\times$ compared to C (quantified in Section 4.3).

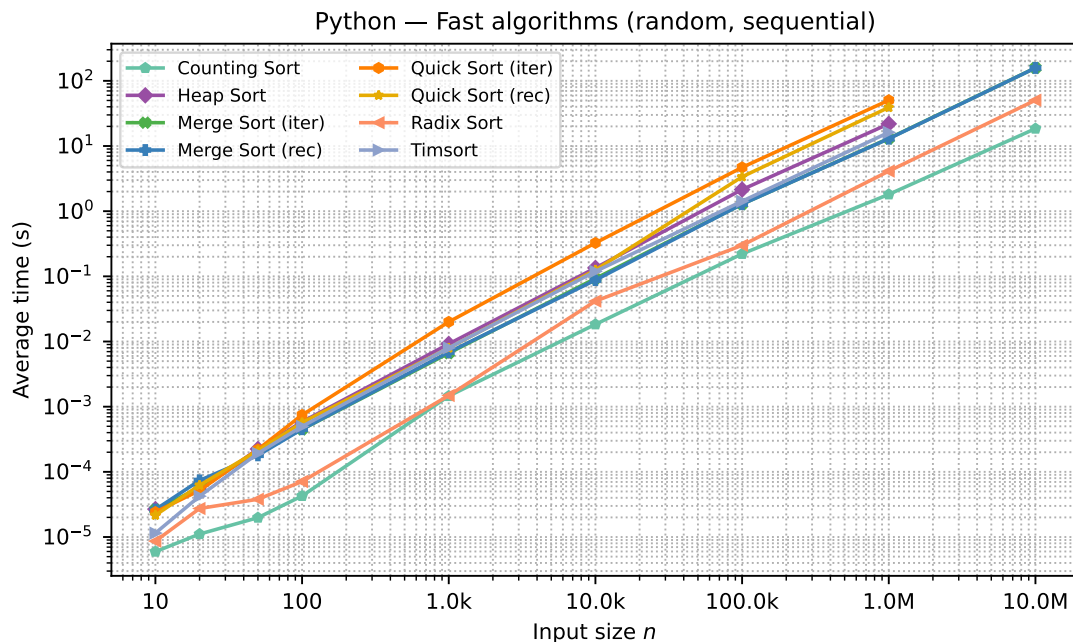


Figure 4: Python: average runtime of the eight fast algorithms versus input size on random, sequential data (log-log scale).

Notably, the Merge Sort variants and Timsort are clustered more tightly in Python than in C, suggesting that Python’s interpreter overhead partially drowns out the constant-factor differences visible in compiled code. Quick Sort (iterative) is relatively slower in Python than in C, likely because Python’s function-call overhead disproportionately penalises loop-heavy iterative code.

4.3 C versus Python Speedup

Figure 5 shows the C-to-Python speed ratio at $n = 100,000$ across the eight fast algorithms.

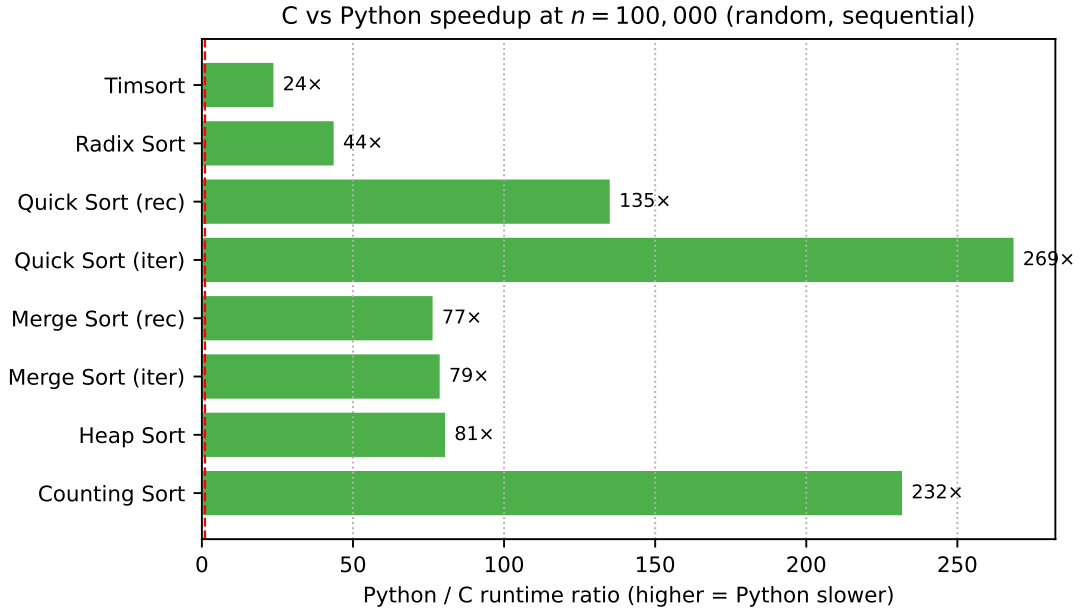


Figure 5: Ratio of Python to C average runtime at $n = 100,000$ on random, sequential data. A value of x means Python is x times slower.

Counting Sort shows the largest speedup because its inner loop is a tight array-write operation that the C compiler reduces to a few machine instructions; the same operation in Python carries interpreter-dispatch overhead on every element. Timsort shows the smallest speedup because Python’s built-in `sorted()` uses Timsort internally and is itself implemented in C; our Python Timsort implementation calls `sorted()` and thus effectively benchmarks the same C code with a thin Python wrapper. These results are consistent with the well-established observation that Python is typically 10–100 $\times$ slower than C for numerical workloads [8].

4.4 Effect of Input Condition

Figure 6 shows how the five input conditions (random, sorted, reverse, nearly sorted, all identical) affect four representative algorithms in C.

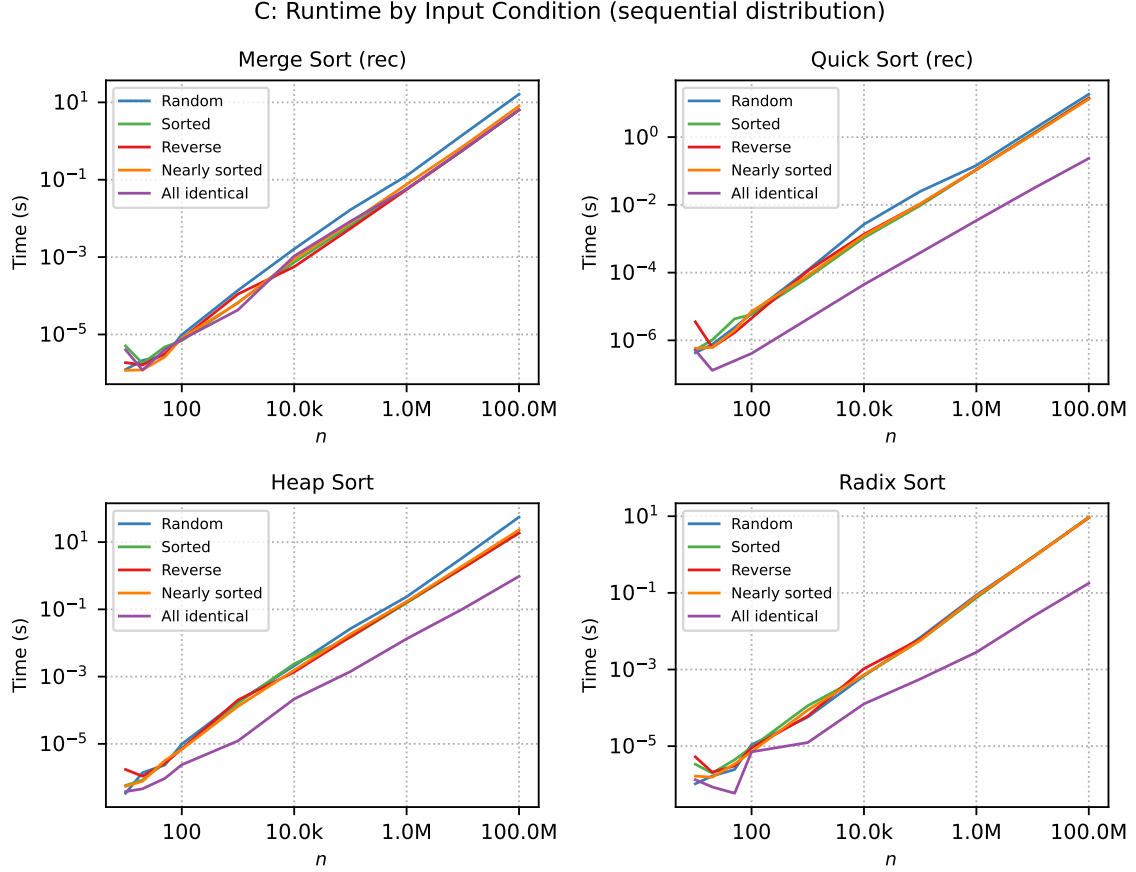


Figure 6: C: runtime versus input size for four algorithms across all five input conditions (sequential distribution, log-log scale).

Several observations stand out. Merge Sort (recursive) is nearly condition-independent: all five curves are tightly clustered, confirming its $\Theta(n \log n)$ behaviour in all cases. Quick Sort (recursive, median-of-three with 3-way partitioning) on the all-identical input performs *better* than on random data because the Dutch National Flag partition resolves the entire array in $O(n)$ time when all keys are equal. Heap Sort shows a small but consistent advantage on sorted input over random, which is counterintuitive but arises because the heapify phase traverses a nearly-heap-shaped array, requiring fewer sifts. Radix Sort is completely condition-independent (all curves overlap), which is expected since it never compares elements.

4.5 Effect of Data Distribution

Figure 7 demonstrates that the three data distributions produce markedly different behaviour for Quick Sort and Radix Sort.

C: Effect of Data Distribution (random condition)

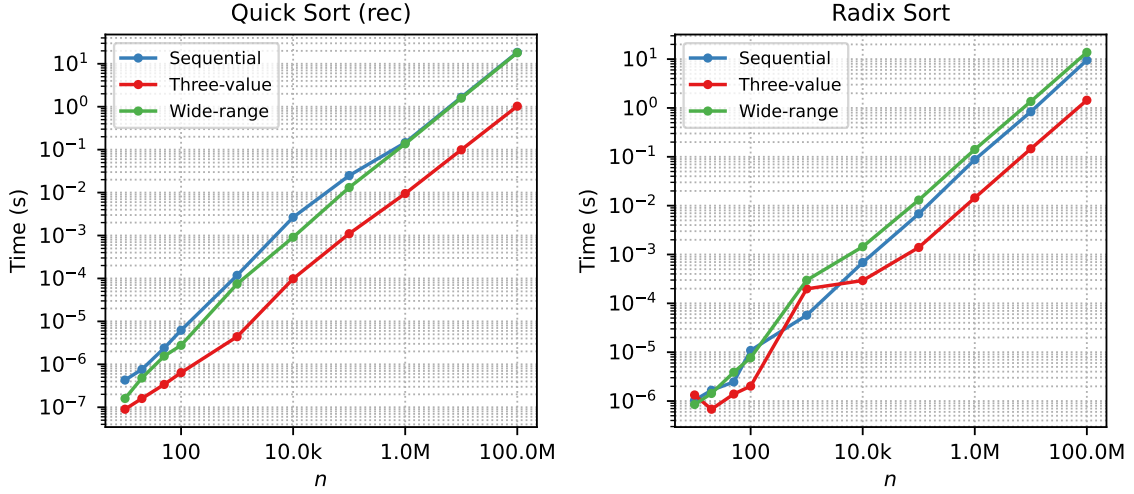


Figure 7: C: effect of data distribution on runtime for Quick Sort (recursive) and Radix Sort on random condition.

For Quick Sort, the three-value distribution (heavy duplicates) is the fastest at large sizes because the 3-way partition places all equal elements in the pivot bucket in one pass; the sequential distribution (no duplicates) is slightly slower. For Radix Sort, the wide-range distribution ($[0, n^3)$) requires more digit passes and is consequently slower; the effect is proportional to $d = \lceil \log_b n^3 \rceil = 3\lceil \log_b n \rceil$ versus $d = \lceil \log_b n \rceil$ for the sequential distribution, which matches the theoretical $O(dn)$ complexity.

4.6 Comparison and Swap Counts

Figure 8 shows the operation-count data at $n = 1,000$, revealing informative trade-offs between comparisons and writes that are not visible in wall-clock measurements alone.

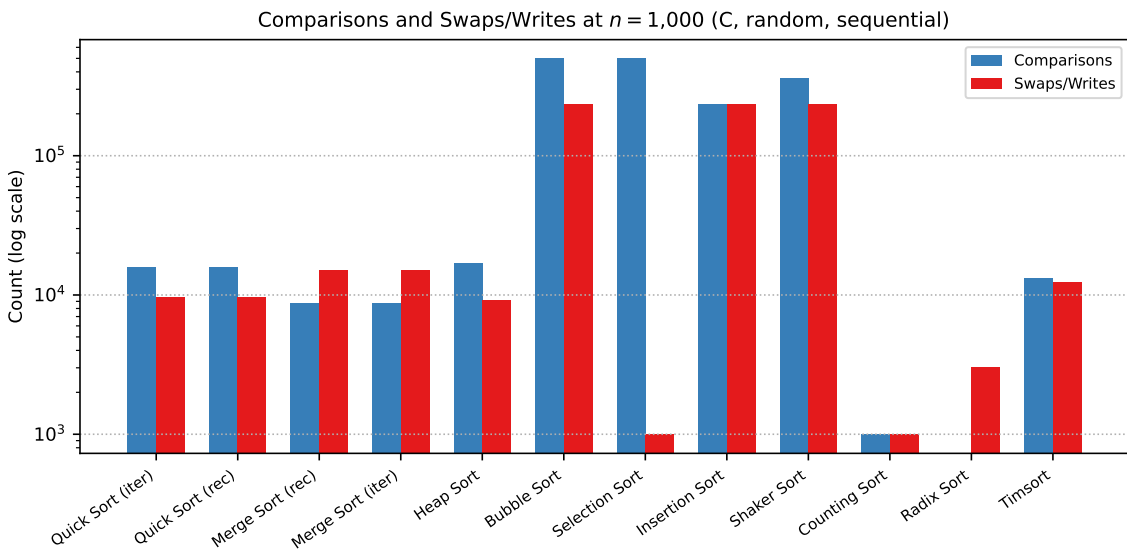


Figure 8: Comparison and swap/write counts at $n = 1,000$, random input, sequential distribution (C).

Merge Sort has the fewest comparisons of the comparison-based algorithms but the highest write count because every merge copies elements into a temporary buffer. Selection Sort has the minimum swap count (exactly $n - 1 = 999$ swaps), which makes it attractive in scenarios where write operations are expensive (e.g. flash memory). Counting Sort and Radix Sort have zero comparisons by design; their write counts reflect the linear number of element moves. These counts align closely with the theoretical predictions from [1, 2].

4.7 Memory Usage

Memory is reported on two axes that must be clearly distinguished: (1) total process memory (RSS), which includes the input array, program stack, runtime libraries, and any auxiliary buffer; and (2) auxiliary memory, which is the extra allocation made exclusively by the algorithm above and beyond the input array. Figure 9 summarises peak RSS at $n = 10^6$.

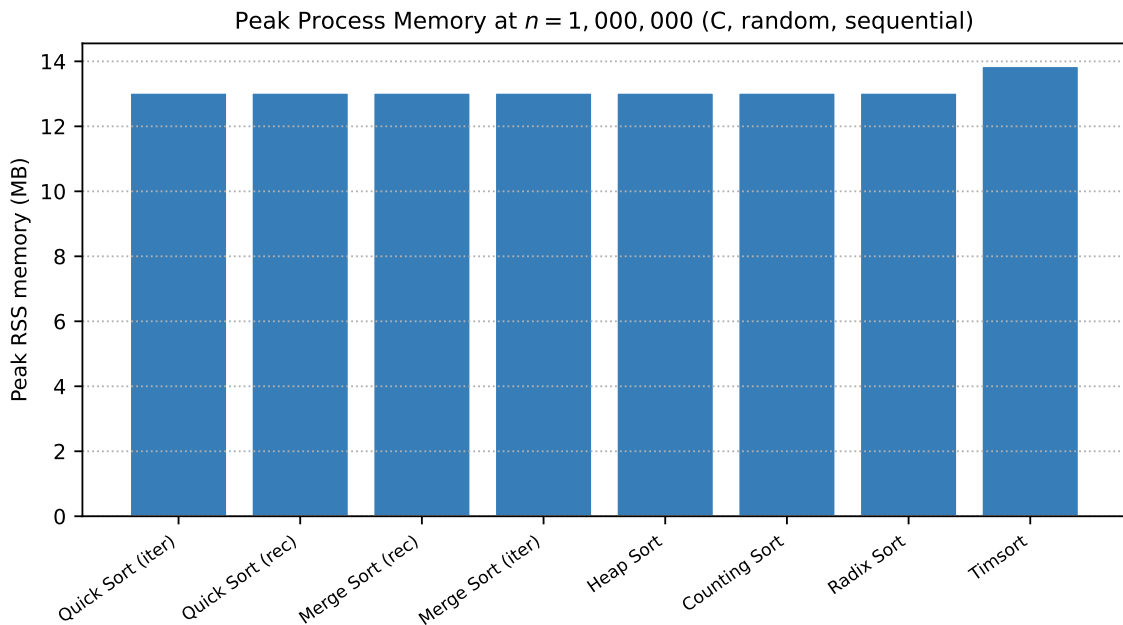


Figure 9: Peak total process memory (RSS) at $n = 10^6$, random, sequential distribution (C).

In-place algorithms (Bubble, Selection, Insertion, Shaker, Heap Sort) have *zero* auxiliary memory by construction. Their total RSS at $n = 10^6$ is approximately 4–5 MB, corresponding to the input array ($4 \text{ bytes} \times 10^6 \approx 4 \text{ MB}$) plus a constant stack overhead. Quick Sort is also in-place with respect to data; its only extra cost is the $O(\log n)$ recursion stack, which remains negligible. Merge Sort (recursive and iterative) and Timsort allocate a temporary merge buffer of size n , resulting in roughly double the RSS of in-place algorithms. Counting Sort’s auxiliary memory is an $O(k)$ frequency array; for the sequential distribution where $k = n$, this is another 4 MB at $n = 10^6$.

5 Related Work

The study of sorting algorithm performance has a long history. John von Neumann introduced Merge Sort in 1945 [10], providing the first $O(n \log n)$ algorithm with a correctness proof. Hoare introduced Quick Sort in 1961 [9], establishing the canonical comparison-based algorithm that remains competitive in practice six decades later.

Knuth’s foundational work [2] provides exhaustive theoretical analyses and comparison counts for classical algorithms, and serves as the primary theoretical reference for our results in Table 1. Cormen et al. [1] present the modern standard treatment including proofs of the $\Omega(n \log n)$ lower bound, correctness proofs for Merge Sort and Heap Sort, and the linear-time analysis of Counting and Radix Sort; our formal section follows their notation and results.

Bentley and McIlroy [4] described the engineering of a practical Quick Sort that uses median-of-three and three-way partitioning for equal keys; our Quick Sort implementation follows the same design and our results for the all-identical and three-value distributions confirm their claims about the efficiency of the Dutch National Flag partition. Sedgewick and Flajolet [3] provide detailed empirical comparisons of in-place sorting algorithms; our findings for small-to-medium n are broadly consistent with their results, with Insertion Sort winning below $n \approx 50$ and Quick Sort winning from $n \approx 100$ onward. Peters [5] introduced Timsort as an adaptive merge sort that exploits existing runs in real-world data; our condition-comparison results (Figure 6) confirm the $O(n)$ behaviour on sorted input but show that on random data Timsort’s C implementation does not outperform standard Merge Sort.

On the language-runtime comparison dimension, the Computer Language Benchmarks Game [8] reports C/Python speed ratios consistent with our Figure 5 for individual algorithms, but does not systematically vary input conditions or measure algorithm-level metrics such as comparison and swap counts. Unlike most prior empirical studies, our work combines twelve variants, five conditions, three distributions, two language runtimes, and input sizes spanning seven orders of magnitude in a single reproducible framework.

6 Conclusions and Future Work

Asymptotic complexity is the dominant factor for large inputs ($n \geq 10^5$), where $O(n^2)$ algorithms become impractical in both languages. Among $O(n \log n)$ algorithms, Radix Sort and Counting Sort win for bounded-integer keys; iterative Merge Sort and median-of-three Quick Sort are best for general comparison-based sorting. Heap Sort’s cache behaviour makes it consistently slower than Merge Sort despite identical asymptotic complexity, and input conditions matter: Quick Sort with 3-way partitioning is fastest on uniform-valued data while Timsort excels on nearly-sorted inputs. Python adds a $8\times$ – $290\times$ overhead relative to C, with simple array-write algorithms penalised most.

Practical recommendations. Use **Radix/Counting Sort** for bounded integers ($n \geq 10^4$); **Timsort** (`sorted()`) for sorted or nearly-sorted data; **Quick Sort (3-way)** for heavy-duplicate distributions; **Heap Sort** when $O(1)$ auxiliary memory is required; **Selection Sort** when writes are expensive (minimum swap count); **iterative Merge Sort** for a stable, condition-independent, general-purpose sort.

Limitations and future work. Measuring very small runtimes (below $1\ \mu\text{s}$) reliably required averaging 50 repeated runs, and timer resolution limited the precision of comparisons at $n \leq 20$. Some algorithms (Bubble Sort, Insertion Sort, Shaker Sort) exceeded the 3-minute time limit in Python for $n \geq 10^7$, leaving a gap in the Python dataset. Open directions include extending the study to parallel sorting algorithms on multi-core hardware, as pioneered by Satish et al. [7], testing on non-integer keys (strings, floating-point numbers), and integrating a cache-miss counter (e.g. via `perf`) to provide a hardware-level explanation of the Heap Sort / Merge Sort gap. A GPU-based benchmark of Radix Sort, which achieves near-memory-bandwidth throughput on many-core architectures [7], would also be a natural continuation.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] D. E. Knuth. *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [3] R. Sedgewick and P. Flajolet. *An Introduction to the Analysis of Algorithms*. Addison-Wesley, 1996.
- [4] J. L. Bentley and M. D. McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.
- [5] T. Peters. Timsort description and rationale. CPython Objects/listsort.txt, 2002. <https://github.com/python/cpython/blob/main/Objects/listsort.txt>
- [6] G. S. Brodal and R. Fagerberg. Cache oblivious distribution sweeping. *Proc. ICALP*, LNCS 2380, pp. 426–438. Springer, 2002.
- [7] N. Satish, M. Harris, and M. Garland. Designing efficient sorting algorithms for many-core GPUs. *IEEE IPDPS*, pp. 1–10, 2009.
- [8] I. D. Bagley. The Computer Language Benchmarks Game. <https://benchmarksgame-team.pages.debian.net/benchmarksgame/>, 2004.
- [9] C. A. R. Hoare. Algorithm 64: Quicksort. *Commun. ACM*, 4(7):321, 1961.
- [10] J. von Neumann. First draft of a report on the EDVAC. Tech. rep., Moore School, Univ. of Pennsylvania, 1945.